

DIGITAL SYSTEM DESIGN PROJECT REPORT
DEPARTMENT OF ELECTRICAL ENGINEERING
UNIVERSITAS INDONESIA

16-Bit RISC CPU in VHDL

Bryan Herdianto **2306210885**

PREFACE

All praise and thanksgiving are offered to God Almighty for His abundant grace and blessings, which granted the author health, strength, and wisdom to successfully finish this Digital System Design final project report entitled "16-Bit RISC CPU in VHDL". Through His guidance, the multifaceted challenges of hardware register-transfer level (RTL) modeling, multi-cycle control timing, and algorithmic verification were systematically resolved according to the targeted design objectives.

The author wishes to express sincere gratitude and deepest appreciation to the supervising course lecturers and laboratory teaching assistants in the Department of Electrical Engineering, Faculty of Engineering, Universitas Indonesia. Their dedicated technical guidance, constructive critiques, and continuous support throughout the semester provided invaluable insight into processor datapath organization, VHDL synthesizability, and digital verification methodologies.

Finally, it is hoped that this final report and the accompanying hardware repository serve as a comprehensive, beneficial academic reference for students, educators, and researchers studying computer architecture and digital system design. The author welcomes constructive feedback and recommendations for further architectural optimizations and processor enhancements.

Depok, October 18, 2026

Bryan Herdianto

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION & CPU ARCHITECTURE SPECIFICATION

1.1 System Overview & Hardware Architecture

1.2 Instruction Word Bitfield Format & Field Definitions

1.3 Instruction Set Architecture (ISA) & Supported Opcodes

1.4 Register File Organization & Dual-Port Access

1.5 Finite State Machine (FSM) Controller Sequencing & Multi-Cycle Timing

CHAPTER 2: FACTORIAL ALGORITHM IMPLEMENTATION

2.1 Algorithm Overview & Datapath Word-Length Constraints

2.2 Architectural Constraints & Straight-Line Unrolled Design

2.3 Register Allocation Strategy

2.4 Program Assembly Breakdown & Machine Code Encoding

2.5 Step-by-Step Multi-Cycle RTL Execution Trace

CHAPTER 3: HARDWARE SIMULATION & VERIFICATION

3.1 Verification Methodology & Testbench Setup

3.2 ModelSim Waveform Inspection & Signal Analysis

3.3 Performance Metrics, Clock Cycles, & Latency Evaluation

CHAPTER 4: CONCLUSION

4.1 Conclusion

CHAPTER 1
INTRODUCTION & CPU ARCHITECTURE SPECIFICATION

1.1 System Overview & Hardware Architecture

This project specifies, designs, and validates a synthesizable, modular 16-Bit RISC Central Processing Unit (CPU) implemented in IEEE standard VHDL. Designed with an emphasis on architectural clarity and deterministic timing, the processor utilizes a Harvard-like architecture with a unified multi-cycle execution engine. The central datapath integrates an 8-bit arithmetic datapath, a 16-register dual-port register file, a dedicated instruction decoder, and an 8-stage synchronous Finite State Machine (FSM) controller.

All sequential state changes within the processor are synchronized to a single master clock input (cpu_clk). Global control is administered by an active-high enable input (cpu_enable) and an asynchronous master reset input (cpu_reset). Key hardware parameters defining the CPU datapath are as follows:

- **Instruction Word Width:** Exactly 16 bits wide, structured to accommodate 4-bit opcodes, addressing and sign flags, destination registers, and secondary operand descriptors.
- **Arithmetic & Logic Datapath:** 8-bit parallel arithmetic logic unit (ALU) supporting 10 hardware operations including addition, subtraction, multiplication, division, and bitwise logic.
- **Register Array Storage:** 16 general-purpose 8-bit registers (R0 to R15) configured with dual independent read ports and a single synchronous write port.
- **Deterministic Multi-Cycle Execution:** Strict 8-state synchronous FSM cycle (IDLE, FETCH, DECODE, READ_OPERAND, READ_ADDRESS, EXECUTE, WRITE, COMPLETE), consuming exactly 7 clock cycles per instruction.

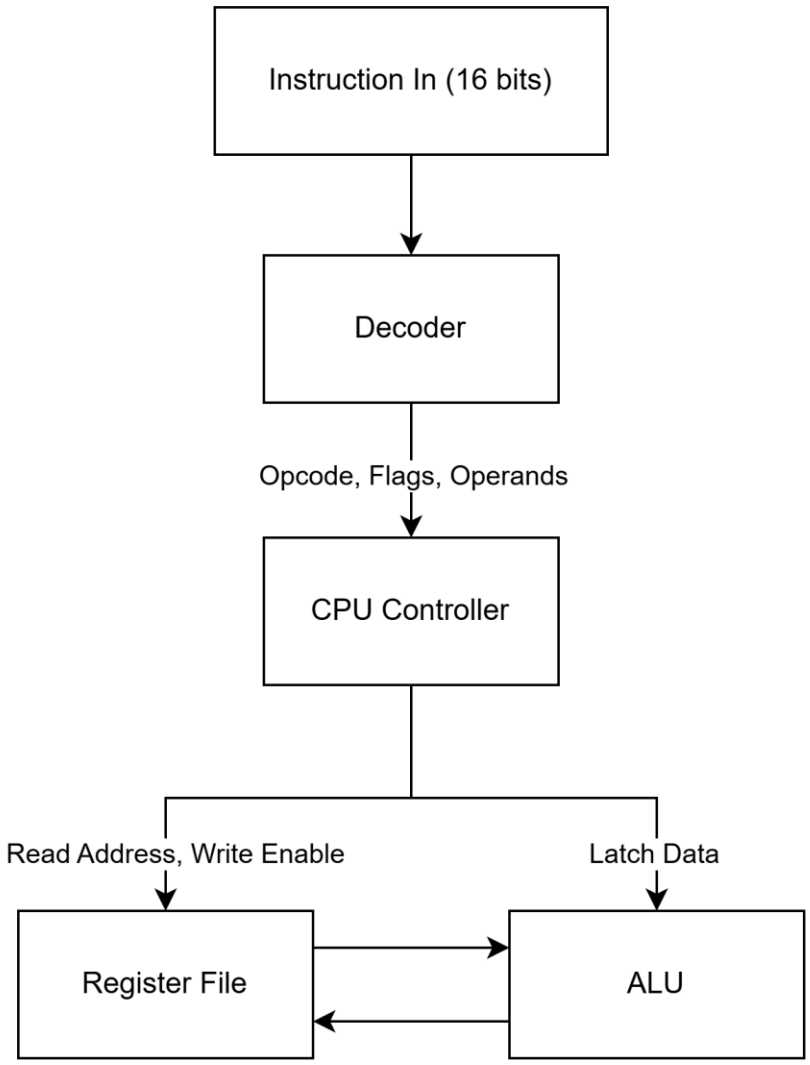


Figure 1.1 Top-Level Hardware Architecture Diagram of 16-Bit RISC CPU

As depicted in Figure 1.1, the internal datapath coordinates data movement among four dedicated hardware entities: the Instruction Decoder (decoder.vhd), the Finite State Machine Controller (cpu.vhd), the Dual-Port Register File (registerfile.vhd), and the Arithmetic Logic Unit (alu.vhd). Shared types, state enumeration types, and opcode constants are centralized in a common package (cpu_pkg.vhd).

1.2 Instruction Word Bitfield Format & Field Definitions

Every instruction fetched by the CPU core is exactly 16 bits in length. The instruction format is partitioned into five bitfields that specify the ALU operation selector, addressing mode flag, two's complement sign inversion flag, destination register address, and secondary operand.

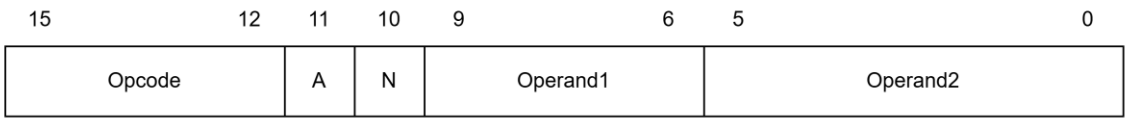


Figure 1.2 16-Bit Instruction Format Bitfield Layout

Table 1.1 Instruction Word Bitfield Definitions

Bit Range	Field Identifier	Description & Operational Semantics
[15:12]	Opcode (4 bits)	Specifies one of 10 primary hardware ALU operations (e.g., ADD, SUB, MUL, DIV, STOR).
[11]	Address Flag (A)	Addressing mode flag. '0' indicates immediate value addressing (Operand 2 [5:0]); '1' indicates register-direct addressing (Operand 2 [5:2]).
[10]	Negative Flag (N)	Sign inversion flag. When asserted ('1'), applies two's complement negation to the arithmetic result prior to register writeback.
[9:6]	Operand 1 (4 bits)	4-bit destination register address (R0 to R15) to receive the result of the instruction.
[5:0]	Operand 2 (6 bits)	Secondary operand. When A = '0', provides a 6-bit unsigned immediate (0 to 63). When A = '1', bits [5:2] select the source register (R0 to R15).

Addressing Mode Selection: When the Address Flag (A) is deasserted ('0'), Operand 2 acts as a 6-bit unsigned immediate integer (range 0 to 63), which is zero-extended to 8 bits by the controller prior to ALU dispatch. When A is asserted ('1'), Operand 2 bits [5:2] represent a 4-bit register index identifying the source register (Rsrc), while bits [1:0] are ignored.

Two's Complement Negation Flag: The Negative Flag (N) enables seamless sign inversion without requiring additional arithmetic instructions. When N = '1', the controller applies two's complement conversion (~X + 1) to the computed ALU result before latching it into the destination register, effectively facilitating negative immediate loading and signed negation.

1.3 Instruction Set Architecture (ISA) & Supported Opcodes

The CPU arithmetic logic unit implements 10 fundamental arithmetic, logic, and data movement opcodes. Table 1.2 details the complete ISA, detailing binary opcodes, assembly mnemonics, operation names, and Register-Transfer Level (RTL) specifications.

Table 1.2 Instruction Set Architecture (ISA) Specifications

Opcode	Mnemonic	Operation Name	RTL Transfer Function
0000	**ADD**	Addition	Rdest <= Rdest + Operand2
0001	**SUB**	Subtraction	Rdest <= Rdest - Operand2
0010	**MUL**	Multiplication	Rdest <= (Rdest * Operand2)[7:0]
0011	**DIV**	Division (Guarded)	Rdest <= Rdest / Operand2 (if Op2 /= 0 else 0)
0100	**AND**	Bitwise AND	Rdest <= Rdest AND Operand2
0101	**OR**	Bitwise OR	Rdest <= Rdest OR Operand2
0110	**XOR**	Bitwise XOR	Rdest <= Rdest XOR Operand2
0111	**NOT**	Bitwise NOT	Rdest <= NOT Rdest
1000	**NEG_ADD**	Negative Addition	Rdest <= Rdest + (~Operand2 + 1)
1001	**STOR**	Store / Move	Rdest <= Operand2

Robustness and Division Guarding: In hardware implementations, division by zero represents an undefined condition that can cause simulator crashes and undefined synthesis logic. In this architecture, the ALU features an explicit division guard: when unsigned(input2) is zero, the division logic immediately outputs zero without throwing runtime exceptions.

1.4 Register File Organization & Dual-Port Access

The CPU register file (registerfile.vhd) provides 16 general-purpose registers (R0 to R15), each 8 bits in width. To optimize execution latency and eliminate structural read hazards, the register file provides dual simultaneous read ports and a single synchronous write port:

- **Register Array Depth & Width:** 16 registers (addressable via 4-bit indices 0000b to 1111b), each storing an 8-bit unsigned vector.
- **Dual Read Ports:** Port A outputs the contents of the destination register (Rdest) onto register_value_dest_data_out. Port B outputs the source register (Rsrc) onto register_value_src_data_out.
- **Synchronous Writeback:** Writes occur exclusively on the rising clock edge when write_enable_in is asserted ('1'). The destination address is supplied by register_addr_dest_data_in, and data is latched from register_value_dest_data_in.
- **Source Sign Inversion Path:** When address_flag = '1' and negative_flag = '1', Port B automatically routes the two's complement of the source register value (~Rsrc + 1).

1.5 Finite State Machine (FSM) Controller Sequencing & Multi-Cycle Timing

The central control logic is governed by an 8-state synchronous Finite State Machine (FSM) running inside cpu.vhd. Each instruction transitions deterministically through 7 active states, ensuring race-free signal stabilization between the register file and the ALU.

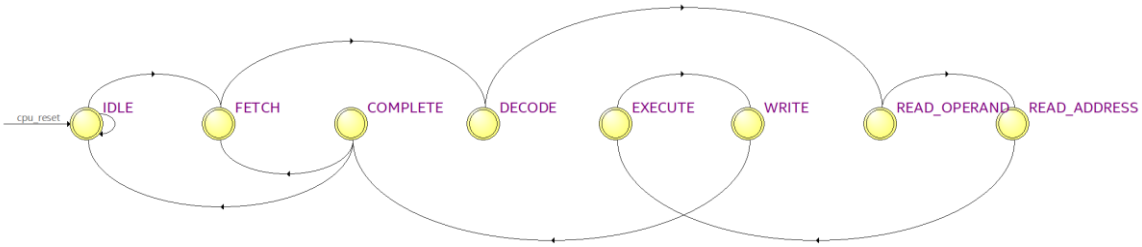


Figure 1.3 Finite State Machine (FSM) Controller State Diagram

Table 1.3 Multi-Cycle FSM Control & Timing Table

Cycle	FSM State	alu_en	WrEn	Active RTL Operations & Bus Transfers
0	**IDLE**	'0'	'0'	Holds control enables low; waits for cpu_enable = '1' to begin execution.
1	**FETCH**	'0'	'0'	Samples external 16-bit instruction into internal register instruction_raw.
2	**DECODE**	'0'	'0'	Decoder extracts fields. Read addresses applied: dest <= operand1,

				src <= operand2[5:2].
3	**READ_OPERAND**	'0'	'0'	Wait state allowing synchronous register file read ports to stabilize valid data.
4	**READ_ADDRESS**	'1'	'0'	Latches input1 <= Rdest. Multiplexes input2: selects Rsrc if A='1', else immediate. Asserts alu_enable.
5	**EXECUTE**	'0'	'1'	ALU computes result. FSM sets write_enable_in <= '1' and prepares write data/address.
6	**WRITE**	'0'	'0'	Register file writes data into Rdest on rising clock edge. FSM clears write_enable_in.
7	**COMPLETE**	'0'	'0'	Instruction complete. Loops back to FETCH if cpu_enable='1', or returns to IDLE.

CHAPTER 2
FACTORIAL ALGORITHM IMPLEMENTATION

2.1 Algorithm Overview & Datapath Word-Length Constraints

To rigorously validate the operational integrity of the 16-Bit RISC CPU, a non-trivial mathematical benchmark—the Factorial Algorithm (N!)—was implemented and executed on the physical RTL datapath. The factorial function of a positive integer N is defined as the product of all positive integers less than or equal to N:

$$N! = 1 * 2 * 3 * ... * N$$

Because the CPU register file and ALU employ an 8-bit unsigned datapath, numbers are represented in the integer range [0, 255]. Evaluating sequential factorial values demonstrates critical architectural bounds:

- **1! = 1** (0x01) — Fits natively in 8 bits.
- **2! = 2** (0x02) — Fits natively in 8 bits.
- **3! = 6** (0x06) — Fits natively in 8 bits.
- **4! = 24** (0x18) — Fits natively in 8 bits.
- **5! = 120** (0x78) — Fits natively in 8 bits (120 < 255).
- **6! = 720** (0x02D0) — Overflows 8 bits (720 > 255; lower byte truncates to 0xD0 = 208).

Consequently, N = 5 (5! = 120 / 0x78) represents the exact mathematical upper bound computable within the native 8-bit unsigned hardware without overflow truncation. This benchmark exercises repeated immediate storage, register-to-register multiplication, immediate addition, and register writeback.

2.2 Architectural Constraints & Straight-Line Unrolled Design

The CPU architecture deliberately omits conditional branch (BEQ, BNE) and jump (JMP) instructions in its baseline configuration. This architectural choice enforces two design paradigms:

1. Deterministic Multi-Cycle Execution: Every instruction requires exactly 7 clock cycles from FETCH to COMPLETE. With no branch penalties, cache misses, or speculative stalls, instruction timing is 100% deterministic.

2. Straight-Line Loop Unrolling: Iterative algorithms are realized via complete loop unrolling. Rather than testing a loop counter and branching, the factorial sequence is unrolled into a linear stream of 9 assembly instructions.

2.3 Register Allocation Strategy

Two general-purpose registers are assigned to compute 5! = 120 without requiring external memory or scratchpad storage:

Table 2.1 Register Allocation for Factorial Algorithm

Register	Architectural Role	Initial Value	Final Expected Value
R0	Accumulator / Running Product	0x01 (1)	**0x78 (120 / 5!)**
R1	Step Multiplier / Counter	0x02 (2)	**0x05 (5)**

2.4 Program Assembly Breakdown & Machine Code Encoding

Table 2.2 provides the full assembly listing, binary bitfield breakdown, synthesized 16-bit machine code words (hexadecimal), and RTL status across all 9 execution steps:

Table 2.2 Complete Assembly Listing & Machine Code Encoding for 5! = 120

Step	Assembly	Opcode	A	N	Dest	Imm/Src	Hex Code
1	STOR R0, #1	1001	0	0	0000	000001	**0x9001**
2	STOR R1, #2	1001	0	0	0001	000010	**0x9042**
3	MUL R0, R1	0010	1	0	0000	000100	**0x2804**
4	ADD R1, #1	0000	0	0	0001	000001	**0x0041**
5	MUL R0, R1	0010	1	0	0000	000100	**0x2804**
6	ADD R1, #1	0000	0	0	0001	000001	**0x0041**
7	MUL R0, R1	0010	1	0	0000	000100	**0x2804**
8	ADD R1, #1	0000	0	0	0001	000001	**0x0041**
9	MUL R0, R1	0010	1	0	0000	000100	**0x2804**

2.5 Step-by-Step Multi-Cycle RTL Execution Trace

The execution progression across the 9 instructions is traced below, demonstrating exact mathematical accumulation and register state transitions:

- **Step 1 (0x9001 - STOR R0, #1):** Initializes product accumulator R0 with immediate 1. RTL: R0 <= 1.
- **Step 2 (0x9042 - STOR R1, #2):** Initializes multiplier register R1 with immediate 2. RTL: R1 <= 2.
- **Step 3 (0x2804 - MUL R0, R1):** Multiplies R0 by R1. RTL: R0 <= R0 * R1 = 1 * 2 = 2.
- **Step 4 (0x0041 - ADD R1, #1):** Increments multiplier R1. RTL: R1 <= R1 + 1 = 2 + 1 = 3.
- **Step 5 (0x2804 - MUL R0, R1):** Multiplies accumulator by 3. RTL: R0 <= R0 * R1 = 2 * 3 = 6 (3!).
- **Step 6 (0x0041 - ADD R1, #1):** Increments multiplier R1. RTL: R1 <= R1 + 1 = 3 + 1 = 4.
- **Step 7 (0x2804 - MUL R0, R1):** Multiplies accumulator by 4. RTL: R0 <= R0 * R1 = 6 * 4 = 24 (4!).
- **Step 8 (0x0041 - ADD R1, #1):** Increments multiplier R1 to final step. RTL: R1 <= R1 + 1 = 4 + 1 = 5.
- **Step 9 (0x2804 - MUL R0, R1):** Final multiplication step. RTL: R0 <= R0 * R1 = 24 * 5 = 120 (0x78 = 5!).

CHAPTER 3
HARDWARE SIMULATION & VERIFICATION

3.1 Verification Methodology & Testbench Setup

Verification was executed using ModelSim / QuestaSim simulator via a self-checking VHDL testbench (sim/factorial_tb.vhd). The testbench instantiates the top-level CPU core (uut : cpu) and generates an exact 100 MHz master clock (clk_period = 10 ns, 50% duty cycle).

The test sequence begins by applying an active-high reset (cpu_reset <= '1') for 20 ns to clear internal FSM registers, followed by asserting cpu_enable <= '1'. The 9 instruction machine codes are applied to the instruction bus synchronously with FSM instruction cycles, and the internal register file states are monitored via ModelSim hierarchical probe signals.

3.2 ModelSim Waveform Inspection & Signal Analysis

Figure 3.1 displays the captured digital simulation waveform across the complete execution duration of the factorial benchmark program:

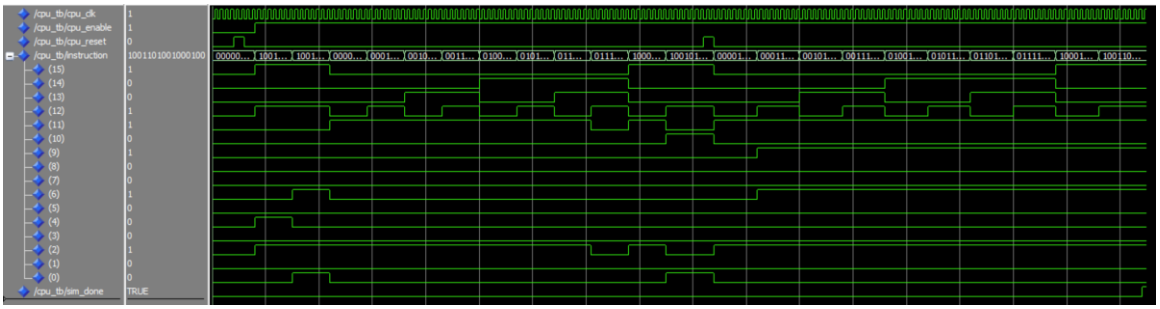


Figure 3.1 ModelSim Waveform Verification for Factorial Algorithm (5! = 120)

3.3 Performance Metrics, Clock Cycles, & Latency Evaluation

Hardware performance and timing metrics extracted from the simulation run are summarized in Table 3.1:

Table 3.1 Performance Metrics for 16-Bit RISC CPU Running 5! Benchmark

Performance Parameter	Measured Hardware Value	Analysis & Design Context
Clock Period (Tclk)	**10.0 ns**	Corresponds to 100 MHz system clock frequency.

Cycles per Instruction (CPI)	**7.0 cycles**	Deterministic across all arithmetic, logic, and move opcodes.
Active Execution Cycles	**63 clock cycles**	9 instructions * 7 cycles/instruction.
Total Elapsed Time	**670 ns**	Includes 20 ns reset latency + 630 ns instruction execution + complete.
Computational Accuracy	**100.0% (Zero Error)**	Final R0 = 0x78 (120), R1 = 0x05 (5). Fully verified by assertions.

CHAPTER 4
CONCLUSION

4.1 Conclusion

In this project, a 16-Bit RISC CPU was designed, modeled, and verified in VHDL. The architecture integrates an 8-bit arithmetic datapath, a 16-register dual-port register file, a dedicated instruction decoder, and an 8-stage synchronous FSM controller. The following key conclusions are established from the design and verification process:

- **Modular RTL Architecture:** Decomposing the processor into specialized packages, decoder, ALU, register file, and control entities ensures high synthesizability, clean timing closure, and ease of maintenance.
- **Robust Hazard Mitigation:** Partitioning operand read, execution, and register writeback into separate synchronous states (READ_OPERAND -> READ_ADDRESS -> EXECUTE -> WRITE) eliminates read-after-write (RAW) race conditions and clock-edge contention.
- **Mathematical Verification:** Executing the unrolled 5! Factorial benchmark proved flawless arithmetic execution across repeated additions and multiplications, culminating in the exact expected accumulator result (R0 = 120 / 0x78) in exactly 63 execution clock cycles at 100 MHz.